DESIGN DOCUMENT · PROPOSAL

Statistics that outlive the page

Optional history of node and server load, player counts and TPS, recorded by each agent, shown as charts, shared with Prometheus, and covered by its own rights.

Decisions taken: collected on the agent · standard retention steps · TPS and MSPT over RCON · player counts only, no play times · dedicated rights split by node and server · Prometheus endpoint included. Builds on `multi-node-design.pdf` and `accounts-rbac-design.pdf`.

1. Summary

Today every number Cubeherd shows is read live: Docker stats per container, host CPU, memory and disk, TPS and MSPT over RCON, the player count from a status ping. Nothing is polled while no page is open, and nothing is kept. That is why an idle Cubeherd costs about 10 MB and 0 % CPU, and also why nobody can answer "was the server slow last night?".

The proposal adds an **opt-in recorder** to the agent. When switched on for a node, the agent samples its host and its running servers once a minute, keeps the samples in a small SQLite file of its own, thins them out over time, and answers history queries from the dashboard. A second, independent feature exports the live values in Prometheus format.

> **Core idea**
> The data is produced on the host that the servers live on, stored there, and fetched only when someone looks at a chart. The dashboard stores nothing and does not need to be running for the history to be complete.

Decisions

QUESTION	DECISION
Where to collect	On the agent, in `metrics.db` next to its other state. The embedded local node does the same through the same code.
Retention	1 minute for 48 hours, 5 minutes for 30 days, 1 hour for 1 year (adjustable per node).
TPS and MSPT	Recorded, one RCON command per running server and minute, only where the software reports them.
Players	Counts only. Names and play times are not stored.
Rights	New rights `stats.server` and `stats.node`, grantable per server and per node like every other right.
Prometheus	Yes: a scrape endpoint on the dashboard with its own token. It works without the recorder.

What stays the same

- Off by default. Without the switch no timer runs, no file is created and nothing changes.
- Independent of the multi-user mode: single-password installs can record statistics too.
- No new dependency (the SQLite driver is already in the binary) and no change to the agent API version: the endpoints are additive.

2. Architecture

```
Dashboard                                  Agent (every node, the embedded one too)
  charts, range picker  -- GET /v1/metrics -->   query, thin out --> metrics.db
  switch, retention  -- configuration push -->   sampler (60 s) --> buffer --> flush (5 min)
  /metrics (Prometheus, live)  <-- views --      hourly: thin out and prune
```

Sampler

- One ticker per agent, started only when recording is on. It never runs while the switch is off.
- Every tick records a row for the node (host CPU, memory, disk, number of running containers) and one per server: `up` (0 or 1), and for servers that are really running also CPU, memory, players and TPS/MSPT.
- **Never disturb a server.** Paused, stopped, starting and busy servers are not queried, so autopause and autostop behave exactly as without the recorder. RCON runs with a short timeout, in parallel with a small limit, and a failed command leaves the field empty rather than failing the tick.
- **CPU from counters.** A single Docker stats call has no previous sample to compare with. The sampler keeps the last cumulative CPU counters of each container and computes the share from the difference; a counter that went backwards (container restarted) is skipped for that tick.
- **Events.** Start, stop, restart, crash and update are written when the manager performs or detects them, independent of the sampling grid.
- **Batching.** Samples are written in one transaction every 5 minutes and on shutdown, which spares SD cards; a crash loses at most the last minutes.

Thinning out and retention

RESOLUTION	KEPT	ROWS PER SERIES (MAX)	VALUES
1 minute	48 hours	2 880	as measured
5 minutes	30 days	8 640	average and maximum
1 hour	1 year	8 760	average and maximum

An hourly job builds the coarser rows from the finer ones (average, maximum, and for TPS the minimum) and deletes what is past retention. A series is the node or one server, so a node with 20 servers holds at most about 21 × 20 000 rows. At roughly 80 bytes per row that is **about 1.6 MB per series and 35 MB for such a node**, an upper bound that is reached after a year. A size cap (default 256 MB) stops writing, and says so in the UI, instead of filling a disk.

Data model

```
samples(res, scope, ts, up, cpu, cpu_max, mem, mem_max, disk, players, players_max,
        tps, tps_min, mspt, mspt_max, n)        PRIMARY KEY (res, scope, ts) WITHOUT ROWID
events(ts, scope, kind, detail)                  -- start, stop, restart, crash, update
meta(key, value)                                 -- schema version, recording since, last prune
```

`scope` is `node` or a server id; time is UTC seconds. `metrics.db` is disposable: if it cannot be opened it is set aside and recreated.

Gaps stay gaps (drawn as breaks). History belongs to the host: it survives removing a node, is not part of the Minecraft backups, and a deleted server keeps its history until retention or a purge.

3. Switching it on, API and screens

Switch

- **Per node:** Nodes → Edit → "Record statistics" with the three retention values and the size cap. The dashboard hands the setting to the agent with the configuration push it already uses (time zone, language, …); the agent stores it in its own settings.
- **Embedded node:** the same fields in the global settings.
- **Agent only:** `CUBEHERD_METRICS=true` for setups that never connect a dashboard; the push may then change it.
- Turning it off keeps the data until it is purged. Switching, changing retention and purging are written to the audit log.

Agent API (additive, API version stays 1)

ENDPOINT	ANSWER
`GET /v1/metrics/node?from&to&points`	Node series, already thinned to at most `points` buckets using the finest resolution that covers the range
`GET /v1/metrics/servers/{id}?from&to&points`	One server's series and its events in the range
`GET /v1/metrics/status`	Recording on or off, since when, size, retention
`DELETE /v1/metrics[/servers/{id}]`	Purge

An agent that predates the endpoints answers 404; the dashboard then shows "update the agent to see history" and the node keeps working otherwise.

Screens

- **Server → Performance:** the live part stays as it is (needs only `server.view`). Below it a range picker (1 h, 24 h, 7 d, 30 d, 1 y) and charts for CPU, memory, players and TPS/MSPT, with start, stop and crash marked on the time axis and the availability for the range.
- **Nodes:** a node's card opens its history: CPU, memory and disk, and the memory share of each server as a stacked chart.
- Charts are server-rendered SVG like the existing sparkline, so no script library and no change to the strict content security policy. Each point carries a `<title>` with its time and value. The range switch swaps the chart fragment through htmx.
- Later: when do people play (heatmap by weekday and hour), a trend that says "disk full in about N days", and CSV export.

4. Rights

Two new permissions, so access to history can differ from access to the live view and can be split by scope like everything else:

PERMISSION	ALLOWS	LOWEST SCOPE
`stats.server`	The history of a server: load, players, TPS, events, availability	server
`stats.node`	The history of a node: CPU, memory, disk, and the share of its servers	node

- Someone with `stats.server` on survival does not see the history of creative; someone with `stats.server` on a node sees all its servers, including later ones. This is the same inheritance as every other right.
- The history needs the right **and** the ability to see the server or node (`server.view`, `nodes.view` is not required for the node history, since it is a right of its own).
- Built-in roles: Owner and Admin get both; Operator gets `stats.server`; Moderator and Viewer get none. Built-in roles are rewritten at start, so existing installs pick this up; custom roles are untouched.
- Switching recording, changing retention and purging need `nodes.manage` (global), because they change a node.
- Single-password mode: everything is allowed, as everywhere.
- Queries are made as the user's request and use the route table: a route without a rule still cannot be registered, and the existing test that calls every guarded route without rights covers the new routes.
- Reading history is not audited (like other reads); a purge, an export and every change of the switch are.

5. Prometheus

For people who already run Prometheus and Grafana, the dashboard offers `GET /metrics` in the text exposition format. It reports **live values** and needs no database, so it works with the recorder off.

METRIC	LABELS
`cubeherd_node_up`, `_cpu_ratio`, `_memory_used_bytes`, `_memory_total_bytes`, `_disk_used_bytes`, `_disk_total_bytes`	node
`cubeherd_server_up`, `_cpu_ratio`, `_memory_used_bytes`, `_players_online`, `_players_max`, `_tps`, `_mspt_milliseconds`	node, server
`cubeherd_build_info`	version

- **Access:** a dedicated scrape token (Settings → Prometheus): shown once, only its hash is stored, sent as `Authorization: Bearer`, can be rotated or switched off. It reads everything and is therefore managed with `settings.manage`; the page says that it bypasses user rights. Creating, rotating and removing are audited. Off by default.
- **No extra load per scrape:** values come from the same short-lived cache as the overview. TPS and MSPT are taken from the last recorded sample, so a scrape never sends RCON commands; without the recorder they are left out.
- Offline nodes report `cubeherd_node_up 0` and no server series.

6. Risks, cost and verification

RISK	MITIGATION
It breaks "nothing runs in the background"	Opt-in per node; the ticker does not exist while off; cost to be measured (target: well under 0.1 % CPU and a few MB for 20 servers)
Sampling wakes or keeps a server alive	Only truly running servers are queried; paused, stopped and starting ones get a status row only
Disk wear and growth	Batched writes, fixed retention, hard size cap, size shown in the UI
Wrong CPU after restarts	Counter deltas, backwards counters skipped, covered by tests
Corrupt database	Set aside and recreated; data is disposable
History hides when people play	Own rights per server and node; counts only, no names
Scrape token leaks	Hash only, rotatable, off by default, audited, documented as global read

Verification plan

- Unit tests with an injected clock: thinning (average, maximum, minimum), retention, gaps, counter reset, size cap, crash-safe flush, concurrent writes and reads under the race detector.
- Agent API tests including an old agent (404) and the dashboard's reaction.
- Rights tests: stats rights per server and per node, inheritance, and the route walk.
- Chart rendering tests (empty, gaps, one point, large ranges) and a Prometheus text-format test against the official parser rules.
- End-to-end in the Lima lab with real agents: record for a while, stop a server, crash one, restart an agent, take the dashboard offline and check that the history has no hole.

7. Phases

Phase 1	**Recorder** Store, sampler, thinning and retention, size cap, agent endpoints, configuration push and switches (node dialog, global settings, environment), embedded node, audit entries.
Phase 2	**Rights and screens** `stats.server` and `stats.node`, server and node history with the range picker, events and availability, "update the agent" hint, purge.
Phase 3	**Prometheus** Scrape token, `/metrics`, documentation with a sample scrape configuration and a Grafana hint.
Later	**Extras** Heatmap of when people play, disk forecast, CSV export, optional alerts (a separate feature).

> **What I would measure first**
> The real cost of one sampling tick with 20 running servers on a small host, the size of a month of data, and how long the first charts take to render over a year of hourly rows. If any of them disappoints, the interval or the stored fields change before anything else.